

DLX Pipeline Hazards

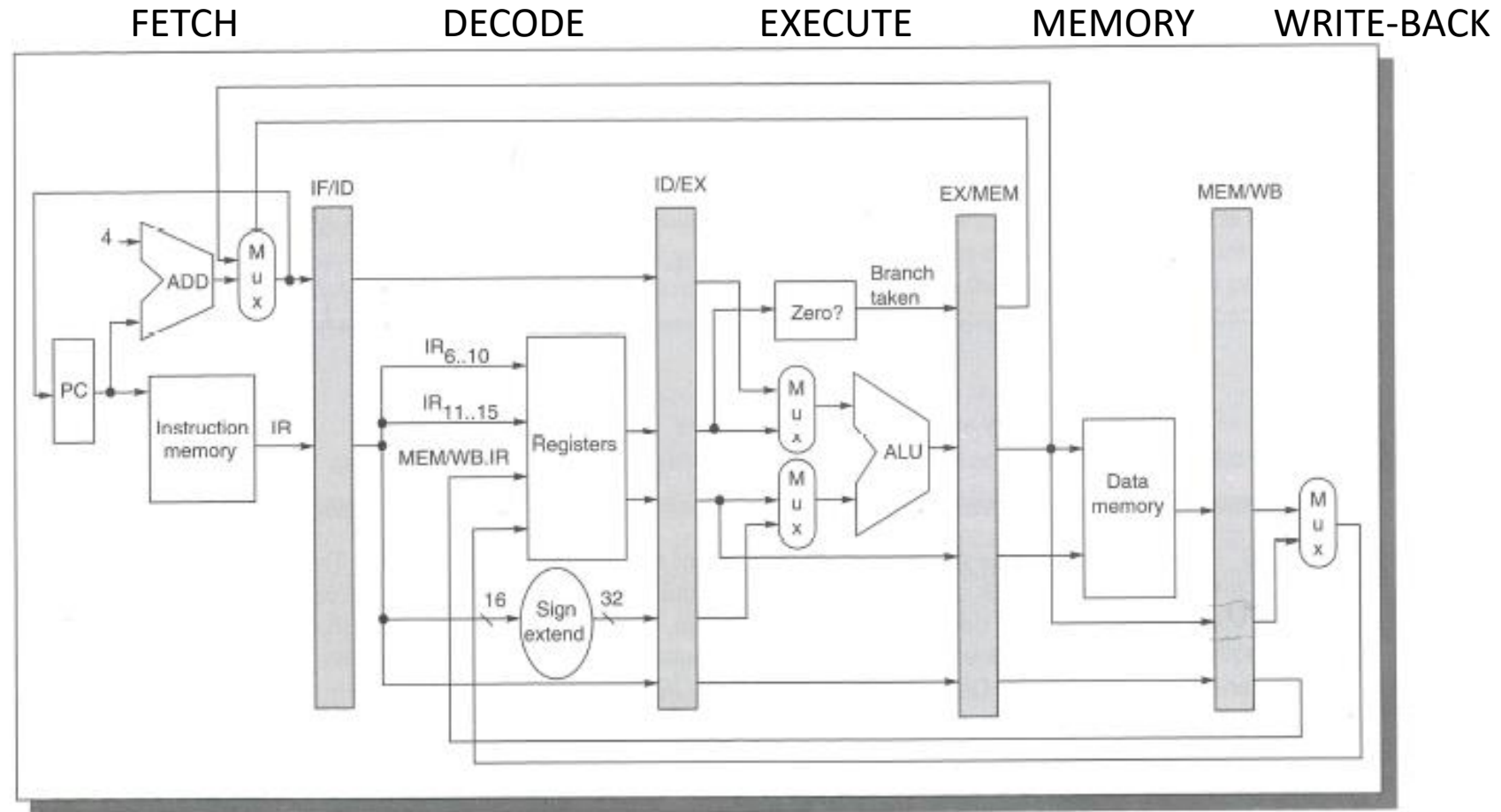
Dr. Jonathan Phillips

Utah State University

ECE 6930 – Advanced Reconfigurable Computing

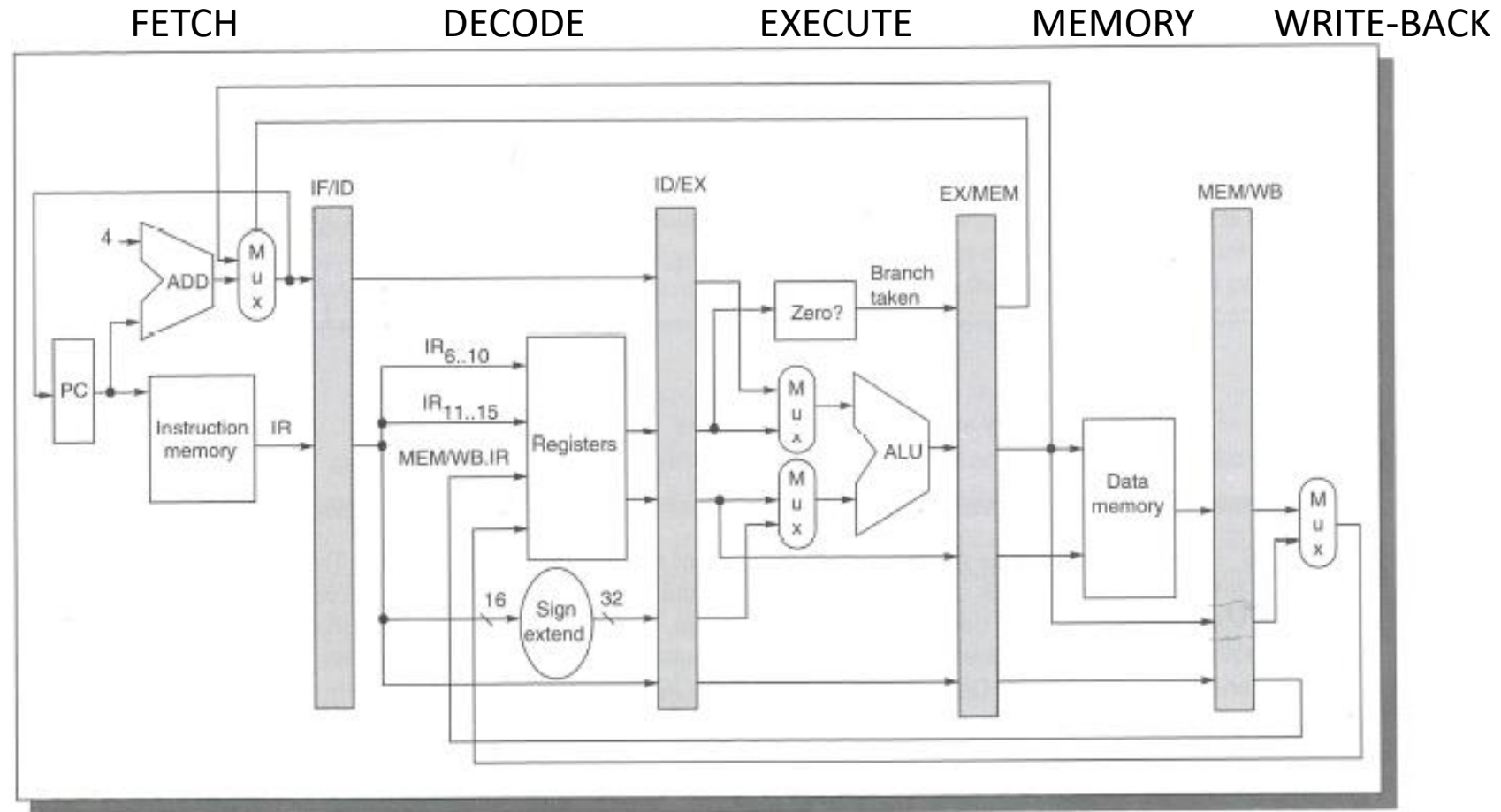
Overview

- Now that all 5 stages exist, is the DLX processor fully functional?
 - No, we need to account for PIPELINE HAZARDS
 - Data Hazards
 - Control Hazards



Overview

- Now that all 5 stages exist, is the DLX processor fully functional?
 - No, we need to account for PIPELINE HAZARDS
 - Data Hazards
 - Control Hazards
- Expecting the programmer to insert NOPs is not reasonable



Data Hazards

- Hennessy & Patterson: “Data hazards arise when an instruction depends on the results of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline.”

Data Hazards

- Hennessy & Patterson: “Data hazards arise when an instruction depends on the results of a previous instruction in a way this is exposed by the overlapping of instructions in the pipeline.”
- Data hazards create the need to STALL the pipeline.

Data Hazard Classification

- RAW (read after write)
 - Most common type of data hazard

Data Hazard Classification

- RAW (read after write)
 - Most common type of data hazard
 - Example:

```
ADDI    R10, R4, 1
```

```
ADD     R11, R10, R10
```

Data Hazard Classification

- RAW (read after write)
 - Most common type of data hazard

- Example:

```
ADDI  R10, R4, 1
```

```
ADD   R11, R10, R10
```

- The pipeline has to wait for R10 to be written before it can be read

Data Hazard Classification

- RAW (read after write)

- Most common type of data hazard

- Example:

```
ADDI  R10, R4, 1
```

```
ADD   R11, R10, R10
```

- The pipeline has to wait for R10 to be written before it can be read
- How many stalls are needed to mitigate this hazard?

Data Hazard Classification

- WAW (write after write)
 - A later instruction tries to write a register or memory location before an earlier instruction
 - Do we need to account for WAW hazards in our DLX pipeline?

Data Hazard Classification

- WAW (write after write)
 - A later instruction tries to write a register or memory location before an earlier instruction
 - Do we need to account for WAW hazards in our DLX pipeline?
- WAR (write after read)
 - A later instruction tries to write a destination before an earlier instruction is done reading it
 - Do we need to account for WAR hazards in our DLX pipeline?

Minimizing Data Hazard Pipeline Stalls

- Forwarding – Fast-tracking results to minimize RAW pipeline stalls
- Example 1

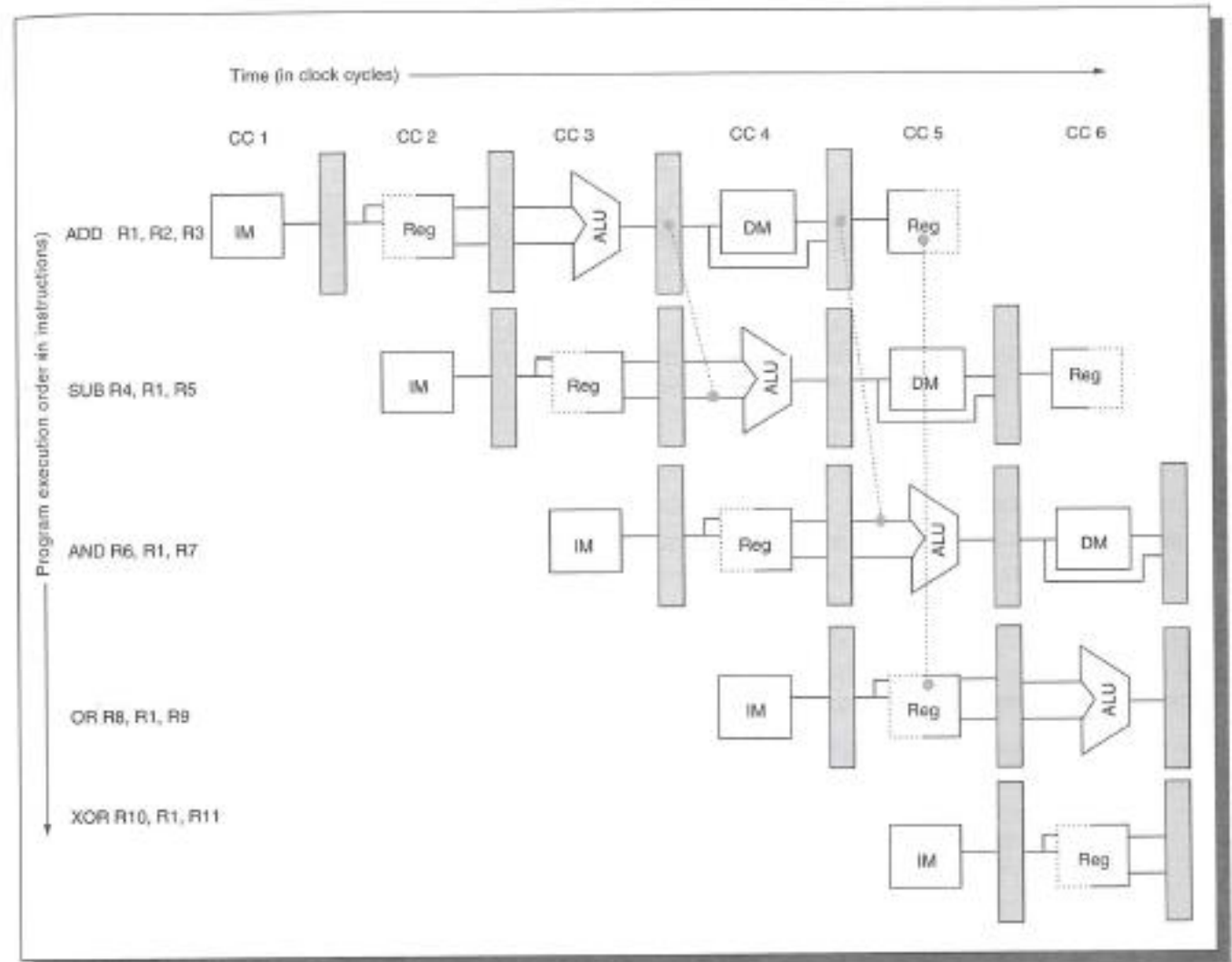


FIGURE 3.10 A set of instructions that depend on the ADD result use forwarding paths to avoid the data hazard. The inputs for the SUB and AND instructions forward from the EX/MEM and the MEM/WB pipeline registers, respectively, to the first ALU input. The OR receives its result by forwarding through the register file, which is easily accomplished by reading the registers in the second half of the cycle and writing in the first half, as the dashed lines on the registers indicate. Notice that the forwarded result can go to either ALU input; in fact, both ALU inputs could use forwarded inputs from either the same pipeline register or from different pipeline registers. This would occur, for example, if the AND instruction was AND R6, R1, R4.

From Computer Architecture A Quantitative Approach, Hennessy & Patterson, 2nd Edition, 1995

Minimizing Data Hazard Pipeline Stalls

- Forwarding – Fast-tracking results to minimize RAW pipeline stalls
- Example 2

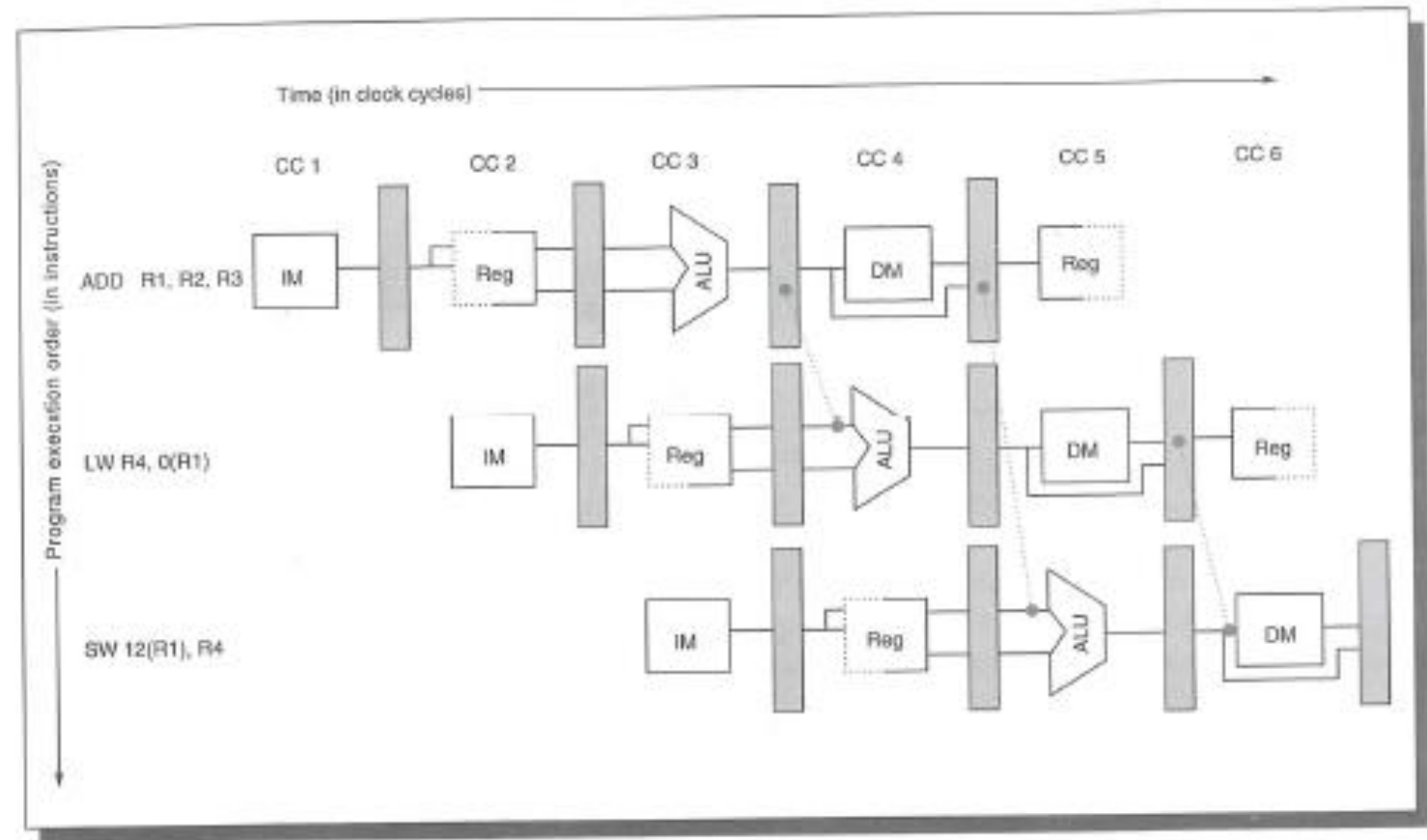


FIGURE 3.11 Stores require an operand during MEM, and forwarding of that operand is shown here. The result of the load is forwarded from the memory output in MEM/WB to the memory input to be stored. In addition, the ALU output is forwarded to the ALU input for the address calculation of both the load and the store (this is no different than forwarding to another ALU operation). If the store depended on an immediately preceding ALU operation (not shown above), the result would need to be forwarded to prevent a stall.

Minimizing Data Hazard Pipeline Stalls

- Forwarding – Fast-tracking results to minimize RAW pipeline stalls
- Example 3

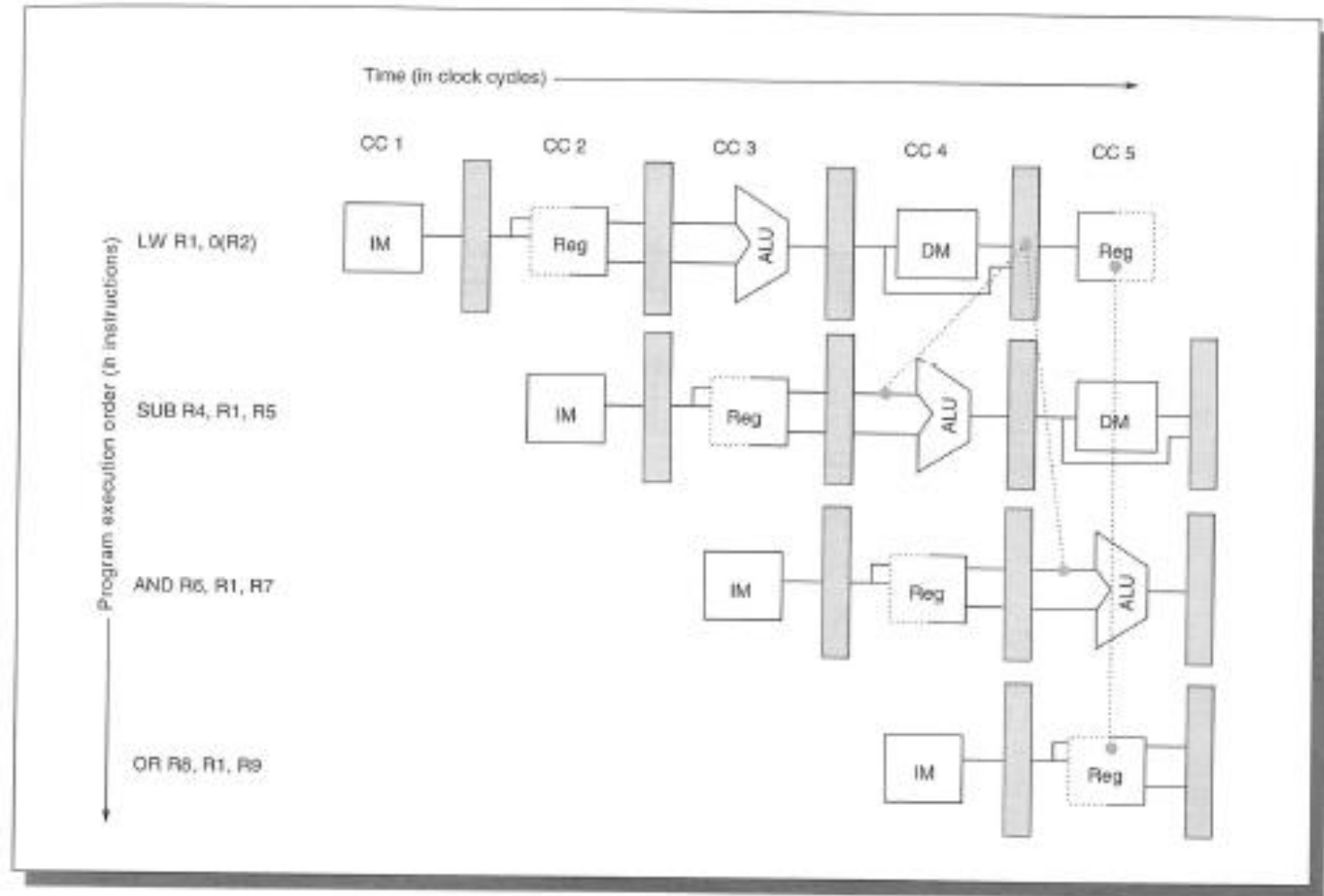


FIGURE 3.12 The load instruction can bypass its results to the **AND** and **OR** instructions, but not to the **SUB**, since that would mean forwarding the result in "negative time."

Minimizing Data Hazard Pipeline Stalls

- Forwarding – Fast-tracking results to minimize RAW pipeline stalls
- Example 3 Solution

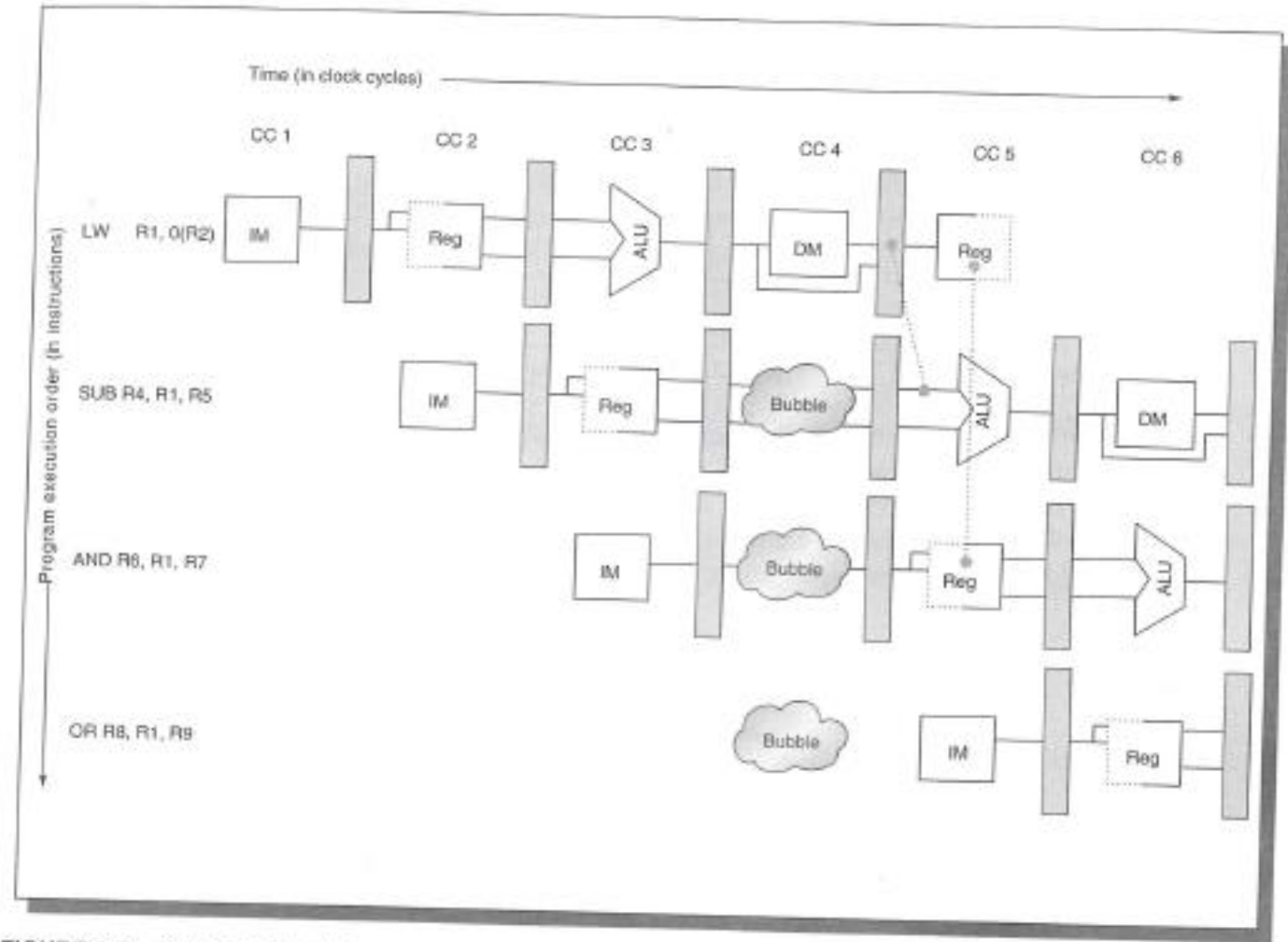


FIGURE 3.13 The load interlock causes a stall to be inserted at clock cycle 4, delaying the *sub* instruction and those that follow by one cycle. This delay allows the value to be successfully forwarded on the next clock cycle.

DLX Forwarding Rules for Data Hazard Mitigation

Source instruction location	Source opcode	Destination instruction location	Destination opcode	Destination of forwarded result	Comparison (if equal then forward)
EX/MEM	ALU	ID/EX	Register-register ALU, immediate ALU, load, store, branch	Top ALU input	EX/MEM(Rd) = ID/EX(Rs1)
EX/MEM	ALU	ID/EX	Register-register ALU	Bottom ALU input	EX/MEM(Rd) = ID/EX(Rs2)
MEM/WB	ALU	ID/EX	Register-register ALU, immediate ALU, load, store, branch	Top ALU input	MEM/WB(Rd) = ID/EX(Rs1)
MEM/WB	ALU	ID/EX	Register-register ALU	Bottom ALU input	MEM/WB(Rd) = ID/EX(Rs2)
MEM/WB	Load	ID/EX	Register-register ALU, immediate ALU, load, store, branch	Top ALU input	MEM/WB(Rd) = ID/EX(Rs1)
MEM/WB	Load	ID/EX	Register-register ALU	Bottom ALU input	MEM/WB(Rd) = ID/EX(Rs2)

DLX Architecture with Data Forwarding

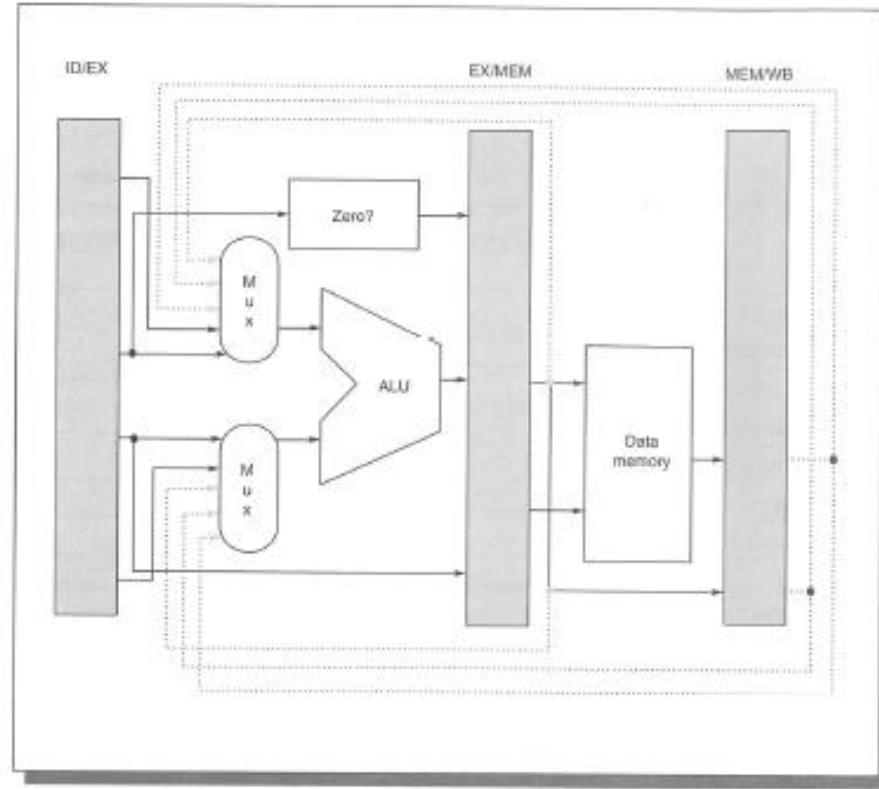


FIGURE 3.20 Forwarding of results to the ALU requires the addition of three extra inputs on each ALU multiplexer and the addition of three paths to the new inputs. The paths correspond to a bypass of (1) the ALU output at the end of the EX, (2) the ALU output at the end of the MEM stage, and (3) the memory output at the end of the MEM stage.

From Computer Architecture A Quantitative Approach, Hennessy & Patterson, 2nd Edition, 1995

Data Hazard Summary

- Forwarding can prevent some data hazards but not all
- A way to “pause” the pipeline must be created
- Is the “pause” as simple as not incrementing the PC?

Control Hazards

- Hennessy & Patterson: “Control hazards arise from the pipelining of branches and other instructions that change the PC.”
- Control Hazards can cause greater performance loss than data hazards.

- Example:

```
J      label
SUBI   R20, R10, 5
ADDI   R20, R20, 1
SLL    R20, R20, R17
NOP
```

- How many cycles of stall does this sequence of instructions incur?

How to handle branches

- Ideas

1. Stall the pipeline on every branch – simple but costly
2. Always assume branch not taken
 1. Flush pipeline if assumption incorrect.
 2. How often will the assumption be correct?
3. Always assume branch taken
 1. Flush pipeline if assumption incorrect.
 2. How often will the assumption be correct?
4. Branch prediction
 1. Implies code profiling

- How long can “bad” instructions live in the pipeline?